

DAY 4

PostgreSQL 18 Backup & Disaster Recovery

Step-by-Step Lab Manual for a LIVE Patroni / Zalando Postgres Operator Cluster

Namespace	postgres-ha
Cluster	acid-postgres-ha (Zalando Operator + Patroni)
PostgreSQL	18
Backup tools	pg_dump, pg_dumpall, pg_basebackup, WAL archiving, pgBackRest
Enterprise backup	Full / Incremental / Differential / Compression / Encryption / Retention / PITR
Restore rule	Restore only to isolated recovery storage / separate recovery cluster

CRITICAL SAFETY RULE: Take backups from the live cluster, but never overwrite the active Patroni member PVCs during restore or PITR. Perform physical restore only on isolated recovery storage or a separate recovery cluster.

COPY/PASTE RULE: Every shell command in this corrected edition is a separate paragraph. Run commands one at a time. Do not paste two commands onto the same shell line unless they are intentionally joined with &&.

Table of Contents

- 0. Lab Architecture, Scope and Safety
- 1. Pre-Lab Validation of the Live Patroni Cluster
- 2. Backup Concepts: Logical, Physical, WAL and PITR
- 3. Lab 1 - Logical Backup from the Live Cluster
- 4. Lab 2 - Physical Backup with pg_basebackup (Corrected)
- 5. Lab 3 - WAL Archiving Validation
- 6. Lab 4 - Configure pgBackRest for the Live Cluster
- 7. Lab 5 - Full Backup with pgBackRest
- 8. Lab 6 - Incremental Backup
- 9. Lab 7 - Differential Backup
- 10. Repository, Compression, Encryption and Retention
- 11. Lab 8 - Restore / Full Cluster Recovery
- 12. Lab 9 - Point-in-Time Recovery (PITR)
- 13. Patroni Failover Considerations During Backup
- 14. Validation, Monitoring, Troubleshooting and Cleanup
- 15. RPO / RTO and Suggested Backup Schedule
- 16. Final Coverage Checklist

Implementation note: Zalando/Spilo commonly uses WAL-G/WAL-E for native backup. pgBackRest is not automatically present in all Spilo images. The pgBackRest portion therefore assumes an immutable image that contains a PostgreSQL 18-compatible pgBackRest build and a durable repository. Do not install packages ad hoc into a running production pod and assume they will survive restart.

0. Lab Architecture, Scope and Safety

0.1 What runs live and what is isolated?

- LIVE: acid-postgres-ha remains Patroni-managed and continues serving traffic during backup exercises.
- LIVE BACKUP: logical dumps, pg_basebackup, WAL archive checks and pgBackRest backup commands run against the current primary or an eligible replica.
- ISOLATED RESTORE: physical restore and PITR use separate recovery storage. Never replace files below active PGDATA while Patroni is running.
- REPOSITORY: use durable object storage (GCS/S3) or, for training only, an RWX volume reachable by every member.

0.2 Recovery architecture

```
Application -> acid-postgres-ha Service -> Patroni Primary
Primary -> logical backup / physical backup / WAL archive / pgBackRest repository
Repository + archived WAL -> isolated recovery PGDATA -> validation -> controlled cutover
```

0.3 Why failover is not backup

Patroni protects availability. It does not protect against logical mistakes such as DROP TABLE, accidental DELETE, bad application updates, corruption that propagates to replicas, or retention mistakes. Backup and PITR provide recovery from those events.

1. Pre-Lab Validation of the Live Patroni Cluster

1.1 Set namespace and inspect resources

```
kubectl config set-context --current --namespace=postgres-ha
kubectl get pods -o wide
kubectl get svc
kubectl get postgresql
kubectl get pvc
kubectl get statefulset
```

1.2 Set variables

```
NS=postgres-ha
CLUSTER=acid-postgres-ha
```

1.3 Identify the current primary - do not assume pod-0

```
for p in acid-postgres-ha-0 acid-postgres-ha-1 acid-postgres-ha-2; do echo "=== $p ==="; kubectl -n "$NS" exec "$p" -- curl -s http://127.0.0.1:8008/ | jq -r '"role=" + (.role // "unknown") + "state=" + (.state // "unknown")'; done
```

Exactly one member should report primary/leader. If the cluster is unhealthy, stop the backup lab and repair HA first.

1.4 Set PRIMARY and REPLICA after checking

```
PRIMARY=acid-postgres-ha-0
REPLICA=acid-postgres-ha-1
```

Change both values to match the actual current primary and a healthy replica.

1.5 Validate role, version, PGDATA and LSN

```
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -Atqc "SELECT pg_is_in_recovery(),  
current_database(), version();"
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -Atqc "SHOW data_directory;"
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "SELECT now(), pg_current_wal_lsn(),  
pg_is_in_recovery();"
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "SELECT application_name, client_addr,  
state, sync_state, sent_lsn, write_lsn, flush_lsn, replay_lsn FROM pg_stat_replication;"
```

2. Backup Concepts: Logical, Physical, WAL and PITR

Logical backup (pg_dump/pg_dumpall) stores SQL objects and data and supports database/object-level restore. Physical backup (pg_basebackup/pgBackRest) copies the cluster data files and is the foundation for cluster-level recovery and PITR. WAL archiving continuously preserves transaction history. PITR restores a physical backup and replays archived WAL up to a selected time/LSN/transaction.

2.1 Full vs differential vs incremental

- Full: complete physical backup; establishes an independent backup baseline.
- Differential: changes since the most recent full backup. Restore uses that full + the chosen differential + required WAL.
- Incremental: changes since the immediately preceding backup. Restore can require a chain of dependent backups + required WAL.
- Archived WAL is not a substitute for a base backup; it is the continuous roll-forward stream used after restoring a physical backup.

2.2 RPO and RTO

RPO is acceptable data loss. Frequent, healthy WAL archiving reduces potential data loss. RTO is acceptable service restoration time; it depends on backup size, storage/network speed, number of backup dependencies, WAL replay volume and validation time.

3. Lab 1 - Logical Backup from the Live Cluster

3.1 Create local backup directory

```
mkdir -p backups/logical
```

3.2 Full cluster logical backup

```
kubectl -n "$NS" exec "$PRIMARY" -- pg_dumpall -U postgres --clean --if-exists > "backups/logical/${CLUSTER}-dumpall-$(date +%Y%m%d-%H%M%S).sql"
ls -lh backups/logical/
```

3.3 Custom-format backup of banking database

```
kubectl -n "$NS" exec "$PRIMARY" -- pg_dump -U postgres -Fc -d banking -f /tmp/banking.dump
kubectl -n "$NS" cp "$PRIMARY:/tmp/banking.dump" "backups/logical/banking-$(date +%Y%m%d-%H%M%S).dump"
```

3.4 Verify without restoring

```
BACKUP_FILE=$(ls -lt backups/logical/banking-*.dump | head -1)
echo "$BACKUP_FILE"
pg_restore -l "$BACKUP_FILE" | head -40
SQL_FILE=$(ls -lt backups/logical/${CLUSTER}-dumpall-*.sql | head -1)
head -30 "$SQL_FILE"
```

3.5 Test logical restore into an isolated test database

```
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "DROP DATABASE IF EXISTS banking_restore_test;"
```

```
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "CREATE DATABASE banking_restore_test;"
BACKUP_FILE=$(ls -1t backups/logical/banking-*.dump | head -1)
echo "$BACKUP_FILE"
kubectl -n "$NS" cp "$BACKUP_FILE" "$PRIMARY:/tmp/banking-restore.dump"
kubectl -n "$NS" exec "$PRIMARY" -- pg_restore -U postgres -d banking_restore_test --clean --if-
exists /tmp/banking-restore.dump
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -d banking_restore_test -c "\dt"
```

On a real production system, run restore tests in a dedicated recovery cluster instead of consuming I/O/storage on the production primary.

4. Lab 2 - Physical Backup with pg_basebackup (Corrected)

This section corrects the original manual. pg_basebackup uses the PostgreSQL replication protocol. A normal superuser login is not enough if pg_hba.conf does not allow that user for replication. In this Zalando/Patroni cluster, use the cluster primary Service and the operator-managed replication/standby credentials.

4.1 Verify the primary Service name

```
kubectl -n "$NS" get svc
PRIMARY_SERVICE=acid-postgres-ha
kubectl -n "$NS" exec "$REPLICA" -- getent hosts "$PRIMARY_SERVICE"
```

Do not use -h 127.0.0.1 when the client command is running in a replica pod. 127.0.0.1 means that same replica pod. Do not use -h postgres-ha unless that Service actually exists. For this cluster, the primary Service is acid-postgres-ha.

4.2 Inspect replication state and lag

```
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "SELECT application_name, client_addr,
state, sync_state, sent_lsn, write_lsn, flush_lsn, replay_lsn,
pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn)) AS replay_lag FROM
pg_stat_replication;"
```

4.3 Identify replication-capable roles

```
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "SELECT rolname, rolreplication,
rolcanlogin FROM pg_roles WHERE rolreplication = true;"
```

4.4 Check replication HBA rules

```
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "SELECT line_number, type, database,
user_name, address, auth_method, error FROM pg_hba_file_rules WHERE 'replication' = ANY(database);"
```

Do not add broad rules such as host replication all 0.0.0.0/0 trust. Use the operator-managed replication role and existing HBA policy.

4.5 Find the operator-managed replication/standby secret

```
kubectl -n "$NS" get secrets | grep -Ei "standby|replic"
```

If the cluster exposes the standard Zalando standby credential secret, set it as follows. If the actual secret name differs, substitute the name returned by the previous command.

```
REPL_SECRET=standby.acid-postgres-ha.credentials.postgresql.acid.zalan.do
```

4.6 Load replication username and password into your local shell

```
REPL_USER=$(kubectl -n "$NS" get secret "$REPL_SECRET" -o jsonpath='{.data.username}' | base64 -d)
REPL_PASS=$(kubectl -n "$NS" get secret "$REPL_SECRET" -o jsonpath='{.data.password}' | base64 -d)
echo "$REPL_USER"
```

Do not echo the password during training or production. Keep it only in the shell variable and unset it after the exercise.

4.7 Test ordinary connectivity with the replication user

```
kubectl -n "$NS" exec "$REPLICA" -- env PGPASSWORD="$REPL_PASS" psql -h "$PRIMARY_SERVICE" -p 5432
-U "$REPL_USER" -d postgres -c "SELECT pg_is_in_recovery(), inet_server_addr(),
inet_server_port();"

```

Expected: `pg_is_in_recovery()` = false because acid-postgres-ha routes to the current primary.

4.8 Prepare backup directory in the client pod

```
kubectl -n "$NS" exec "$REPLICA" -- rm -rf /tmp/basebackup
kubectl -n "$NS" exec "$REPLICA" -- mkdir -p /tmp/basebackup

```

4.9 Run pg_basebackup with the correct host and replication user

```
kubectl -n "$NS" exec "$REPLICA" -- env PGPASSWORD="$REPL_PASS" pg_basebackup -h "$PRIMARY_SERVICE"
-p 5432 -U "$REPL_USER" -D /tmp/basebackup -Ft -z -Xs -P

```

Options: `-Ft` = tar format; `-z` = gzip; `-Xs` = stream WAL while the backup runs; `-P` = progress display.

4.10 Verify physical backup files

```
kubectl -n "$NS" exec "$REPLICA" -- ls -lh /tmp/basebackup
kubectl -n "$NS" exec "$REPLICA" -- find /tmp/basebackup -maxdepth 1 -type f -printf "%f %s
bytes\n"

```

Typical tar-format output includes `base.tar.gz` and WAL tar output when WAL streaming is included.

4.11 Copy the backup out to the workstation

```
mkdir -p backups/physical
PHYSICAL_DIR="backups/physical/basebackup-$(date +%Y%m%d-%H%M%S)"
kubectl -n "$NS" cp "$REPLICA:/tmp/basebackup" "$PHYSICAL_DIR"
find "$PHYSICAL_DIR" -maxdepth 2 -type f -ls | head -20

```

4.12 Remove password variable from the shell

```
unset REPL_PASS

```

4.13 Troubleshooting the exact errors seen in this lab

- could not translate host name "postgres-ha": the Service name is wrong or does not exist. Use `kubectl get svc` and set `PRIMARY_SERVICE=acid-postgres-ha`.
- no `pg_hba.conf` entry for replication connection ... user "postgres": the host is reachable, but postgres is not permitted for replication. Use the operator-managed replication/standby role and its Secret.
- no `pg_hba.conf` entry ... SSL encryption / no encryption: the server evaluated SSL and non-SSL HBA paths and neither matched that replication user/source. Do not bypass this with trust; fix role/HBA/authentication correctly.

5. Lab 3 - WAL Archiving Validation

5.1 Inspect archive settings

```
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "SHOW archive_mode;"
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "SHOW archive_command;"
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "SHOW archive_library;"
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -x -c "SELECT * FROM pg_stat_archiver;"
```

5.2 Force a WAL switch and verify progress

```
BEFORE=$(kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -Atqc "SELECT archived_count FROM pg_stat_archiver;")
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "SELECT pg_switch_wal();"
sleep 10
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "SELECT archived_count, failed_count, last_archived_wal, last_archived_time, last_failed_wal, last_failed_time FROM pg_stat_archiver;"
echo "Archived before test: $BEFORE"
```

5.3 Check native Spilo backup tooling before changing archive_command

```
kubectl -n "$NS" exec "$PRIMARY" -- sh -lc 'command -v wal-g || true; command -v wal-e || true; env | grep -E "^(USE_WALG|WALG_|WALE_)" | sed -E "s/(SECRET|KEY|TOKEN|PASSWORD)=.*/\1=REDACTED/''
```

Do not run two competing archive_command implementations without a migration plan. If WAL-G/WAL-E is active, decide whether pgBackRest will replace it or be used only for a controlled training path.

6. Lab 4 - Configure pgBackRest for the Live Cluster

6.1 Record current live image and configuration

```
CURRENT_IMAGE=$(kubectl -n "$NS" get pod "$PRIMARY" -o jsonpath='{.spec.containers[?(@.name=="postgres")].image}')
echo "$CURRENT_IMAGE"
kubectl -n "$NS" get postgresql "$CLUSTER" -o yaml > "${CLUSTER}-before-pgbackrest.yaml"
kubectl -n "$NS" get statefulset "$CLUSTER" -o yaml > "${CLUSTER}-sts-before-pgbackrest.yaml"
```

6.2 Build an immutable Spilo image containing pgBackRest

```
cat > Dockerfile.spilo-pgbackrest <<'EOF'
ARG BASE_IMAGE
FROM ${BASE_IMAGE}
USER root
RUN apt-get update \
  && apt-get install -y --no-install-recommends pgbackrest \
  && rm -rf /var/lib/apt/lists/*
USER postgres
EOF
CUSTOM_IMAGE=registry.example.com/spilo-pgbackrest:pg18
docker build --build-arg BASE_IMAGE="$CURRENT_IMAGE" -f Dockerfile.spilo-pgbackrest -t "$CUSTOM_IMAGE" .
docker push "$CUSTOM_IMAGE"
```

If the current image does not use apt, adapt the image build to its OS. Every Patroni member should use the same immutable image and a PostgreSQL 18-compatible pgBackRest version.

6.3 Create repository encryption secret

```
PGBR_CIPHER_PASS=$(openssl rand -base64 48)
kubectl -n "$NS" create secret generic pgbackrest-repo-secret --from-literal=repo-cipher-pass="$PGBR_CIPHER_PASS"
unset PGBR_CIPHER_PASS
```

6.4 Discover actual PGDATA for pg1-path

```
PGDATA_ACTUAL=$(kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -Atqc "SHOW data_directory;")
echo "$PGDATA_ACTUAL"
```

6.5 Create pgBackRest configuration

Replace REPLACE_WITH_BUCKET_NAME and the pg1-path value if your discovered PGDATA differs.

```
cat > pgbackrest.conf <<'EOF'
[postgres-ha]
pg1-path=/home/postgres/pgdata/pgroot/data
pg1-port=5432

[global]
repo1-type=gcs
repo1-gcs-bucket=REPLACE_WITH_BUCKET_NAME
repo1-gcs-key-type=auto
repo1-path=/pgbackrest/postgres-ha
repo1-retention-full=2
repo1-retention-diff=4
repo1-cipher-type=aes-256-cbc
compress-type=zst
compress-level=3
process-max=2
start-fast=y
log-level-console=info

[global:archive-push]
process-max=2

[global:archive-get]
process-max=2
EOF
kubectl -n "$NS" create configmap pgbackrest-config --from-file=pgbackrest.conf=./pgbackrest.conf
```

6.6 Patch the PostgreSQL CR during a maintenance window

Use kubectl edit or your GitOps manifest to set the custom image and mount pgbackrest-config at /etc/pgbackrest and pgbackrest-repo-secret at /etc/pgbackrest-secret in the postgres container. Exact CR fields can vary by Postgres Operator version, so preserve the existing CR and validate the generated StatefulSet before proceeding.

```
kubectl -n "$NS" edit postgresql "$CLUSTER"
kubectl -n "$NS" get postgresql "$CLUSTER" -o yaml > "${CLUSTER}-after-pgbackrest.yaml"
kubectl -n "$NS" get pods -w
```

6.7 Validate pgBackRest on every Patroni member

```
for p in $(kubectl -n "$NS" get pod -l cluster-name="$CLUSTER" -o name | cut -d/ -f2); do echo "===
$p ==="; kubectl -n "$NS" exec "$p" -- pgbackrest version; kubectl -n "$NS" exec "$p" -- test -r
/etc/pgbackrest/pgbackrest.conf && echo config-ok; done
```

6.8 Create stanza and validate repository

```
kubectl -n "$NS" exec "$PRIMARY" -- sh -lc 'export PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"; pgbackrest --stanza=postgres-ha stanza-create; pgbackrest --stanza=postgres-ha check'
```

6.9 Configure archive_command only after repository validation

```
kubectl -n "$NS" exec "$PRIMARY" -- patronictl show-config  
kubectl -n "$NS" exec -it "$PRIMARY" -- patronictl edit-config
```

Set `archive_mode=on` and `archive_command` to a shell command that exports `PGBACKREST_REPO1_CIPHER_PASS` from the mounted Secret and executes `pgbackrest --stanza=postgres-ha archive-push %p`. Do not hand-edit `postgresql.conf` inside a Patroni-managed pod.

7. Lab 5 - Full Backup with pgBackRest

```
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "SELECT pg_switch_wal();"
sleep 10
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "SELECT archived_count, failed_count,
last_archived_wal, last_archived_time FROM pg_stat_archiver;"
kubectl -n "$NS" exec "$PRIMARY" -- sh -lc 'export PGBACKREST_REPO1_CIPHER_PASS="$(cat
/etc/pgbackrest-secret/repo-cipher-pass)"; pgbackrest --stanza=postgres-ha --type=full backup'
kubectl -n "$NS" exec "$PRIMARY" -- sh -lc 'export PGBACKREST_REPO1_CIPHER_PASS="$(cat
/etc/pgbackrest-secret/repo-cipher-pass)"; pgbackrest --stanza=postgres-ha info'
```

Expected: the newest backup label ends with F. Record backup start/stop time, database size, repository size and WAL start/stop values.

```
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -d banking -c "CREATE TABLE IF NOT EXISTS
dr_lab_events(id bigint generated always as identity primary key, event_time timestamptz default
clock_timestamp(), note text); INSERT INTO dr_lab_events(note) VALUES ('after full backup'); SELECT
* FROM dr_lab_events ORDER BY id;"
```

8. Lab 6 - Incremental Backup

```
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -d banking -c "INSERT INTO dr_lab_events(note)
VALUES ('before incremental backup');"
kubectl -n "$NS" exec "$PRIMARY" -- sh -lc 'export PGBACKREST_REPO1_CIPHER_PASS="$(cat
/etc/pgbackrest-secret/repo-cipher-pass)"; pgbackrest --stanza=postgres-ha --type=incr backup;
pgbackrest --stanza=postgres-ha info'
```

Expected: the new label ends with I and references an earlier backup set. Incremental = changes since the previous backup.

9. Lab 7 - Differential Backup

```
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -d banking -c "INSERT INTO dr_lab_events(note)
VALUES ('before differential backup');"
kubectl -n "$NS" exec "$PRIMARY" -- sh -lc 'export PGBACKREST_REPO1_CIPHER_PASS="$(cat
/etc/pgbackrest-secret/repo-cipher-pass)"; pgbackrest --stanza=postgres-ha --type=diff backup;
pgbackrest --stanza=postgres-ha info'
```

Expected: the new label ends with D. Differential = changes since the most recent full backup.

10. Repository, Compression, Encryption and Retention

10.1 Repository

The repository stores backup sets, metadata and archived WAL. Object storage is preferred because it is independent of a single Kubernetes node and can be reached by every potential primary.

10.2 Compression

```
compress-type=zst
compress-level=3
```

10.3 Encryption

```
repo1-cipher-type=aes-256-cbc
```

The cipher passphrase is supplied from the Kubernetes Secret via `PGBACKREST_REPO1_CIPHER_PASS`. Losing the passphrase makes encrypted backups unrecoverable.

10.4 Retention

```
repo1-retention-full=2  
repo1-retention-diff=4
```

10.5 Review expiration before deleting backups

```
kubectl -n "$NS" exec "$PRIMARY" -- sh -lc 'export PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"; pgbackrest --stanza=postgres-ha --dry-run expire'  
kubectl -n "$NS" exec "$PRIMARY" -- sh -lc 'export PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"; pgbackrest --stanza=postgres-ha expire'
```

11. Lab 8 - Restore / Full Cluster Recovery

DESTRUCTIVE AT PGDATA LEVEL: Never run this against active acid-postgres-ha PVCs. Create a separate recovery cluster/PVC first.

11.1 Create a separate recovery cluster

```
cat > acid-pgbackrest-recovery.yaml <<'EOF'
apiVersion: acid.zalan.do/v1
kind: postgresql
metadata:
  name: acid-pgbackrest-recovery
  namespace: postgres-ha
spec:
  teamId: acid
  numberOfInstances: 1
  volume:
    size: 20Gi
  postgresql:
    version: "18"
  dockerImage: registry.example.com/spilo-pgbackrest:pg18
EOF
kubectl apply -f acid-pgbackrest-recovery.yaml
```

11.2 Discover recovery StatefulSet/PVC and service account

```
REC_STS=acid-pgbackrest-recovery
kubectl -n "$NS" scale statefulset/"$REC_STS" --replicas=0
kubectl -n "$NS" get pvc | grep "$REC_STS"
REC_PVC=$(kubectl -n "$NS" get pvc -o name | grep acid-pgbackrest-recovery | head -1 | cut -d/ -f2)
echo "$REC_PVC"
POD_SA=$(kubectl -n "$NS" get pod "$PRIMARY" -o jsonpath='{.spec.serviceAccountName}')
echo "$POD_SA"
```

11.3 Create recovery toolbox pod

```
cat > pgbackrest-recovery-toolbox.yaml <<EOF
apiVersion: v1
kind: Pod
metadata:
  name: pgbackrest-recovery-toolbox
  namespace: $NS
spec:
  serviceAccountName: $POD_SA
  restartPolicy: Never
  containers:
    - name: toolbox
      image: registry.example.com/spilo-pgbackrest:pg18
      command: ["sh", "-lc", "sleep infinity"]
      volumeMounts:
        - name: data
          mountPath: /home/postgres/pgdata
        - name: config
          mountPath: /etc/pgbackrest
        - name: secret
          mountPath: /etc/pgbackrest-secret
  volumes:
    - name: data
      persistentVolumeClaim:
        claimName: $REC_PVC
```

```
- name: config
  configMap:
    name: pgbackrest-config
- name: secret
  secret:
    secretName: pgbackrest-repo-secret
EOF
kubectl apply -f pgbackrest-recovery-toolbox.yaml
kubectl -n "$NS" wait --for=condition=Ready pod/pgbackrest-recovery-toolbox --timeout=10m
```

11.4 Discover and prepare recovery PGDATA

```
kubectl -n "$NS" exec pgbackrest-recovery-toolbox -- sh -lc 'find /home/postgres/pgdata -maxdepth 4
-name PG_VERSION -print'
REC_PGDATA=/home/postgres/pgdata/pgroot/data
```

Verify REC_PGDATA before running the next command.

```
kubectl -n "$NS" exec pgbackrest-recovery-toolbox -- sh -lc "mv "$REC_PGDATA" "$
{REC_PGDATA}.before-restore" && mkdir -p "$REC_PGDATA""
```

11.5 Restore the latest backup

```
kubectl -n "$NS" exec pgbackrest-recovery-toolbox -- sh -lc 'export PGBACKREST_REPO1_CIPHER_PASS="$
(cat /etc/pgbackrest-secret/repo-cipher-pass)"; pgbackrest --stanza=postgres-ha
--pg1-path=/home/postgres/pgdata/pgroot/data restore'
```

Correction from the previous manual: do not use `--set=latest`. To restore the latest eligible backup, omit `--set`. Use `--set` only when you have an actual pgBackRest backup label.

11.6 Start and validate the recovery cluster

```
kubectl -n "$NS" delete pod pgbackrest-recovery-toolbox --wait=true
kubectl -n "$NS" scale statefulset/"$REC_STS" --replicas=1
kubectl -n "$NS" rollout status statefulset/"$REC_STS" --timeout=10m
REC_POD=$(kubectl -n "$NS" get pod -l cluster-name=acid-pgbackrest-recovery -o
jsonpath='{.items[0].metadata.name}')
kubectl -n "$NS" exec "$REC_POD" -- psql -U postgres -Atqc "SELECT pg_is_in_recovery(), now();"
kubectl -n "$NS" exec "$REC_POD" -- psql -U postgres -d banking -c "SELECT * FROM dr_lab_events
ORDER BY id;"
```

12. Lab 9 - Point-in-Time Recovery (PITR)

12.1 Take a fresh full backup

```
kubectl -n "$NS" exec "$PRIMARY" -- sh -lc 'export PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"; pgbackrest --stanza=postgres-ha --type=full backup; pgbackrest --stanza=postgres-ha info'
```

12.2 Create a known-good recovery point

```
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -d banking -c "INSERT INTO dr_lab_events(note) VALUES ('PITR good row');"
TARGET_TIME=$(kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -Atqc "SELECT to_char(clock_timestamp() AT TIME ZONE 'UTC', 'YYYY-MM-DD HH24:MI:SS.US') || '+00';")
echo "$TARGET_TIME"
sleep 3
```

12.3 Simulate accidental data loss

```
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -d banking -c "DELETE FROM dr_lab_events;"
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -d banking -c "SELECT count(*) FROM dr_lab_events;"
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "SELECT pg_switch_wal();"
```

12.4 Prepare isolated recovery target again

```
kubectl -n "$NS" scale statefulset/"$REC_STS" --replicas=0
kubectl -n "$NS" wait --for=delete pod/"$REC_POD" --timeout=180s || true
kubectl apply -f pgbackrest-recovery-toolbox.yaml
kubectl -n "$NS" wait --for=condition=Ready pod/pgbackrest-recovery-toolbox --timeout=10m
```

12.5 Restore to target time

Pass TARGET_TIME into the recovery pod with env so the local value is available to the remote shell without unsafe quoting or local command substitution.

```
kubectl -n "$NS" exec pgbackrest-recovery-toolbox -- env TARGET_TIME="$TARGET_TIME" sh -lc 'export PGBACKREST_REPO1_CIPHER_PASS="$(cat /etc/pgbackrest-secret/repo-cipher-pass)"; pgbackrest --stanza=postgres-ha --pg1-path=/home/postgres/pgdata/pgroot/data --type=time --target="$TARGET_TIME" --target-action=promote restore'
```

12.6 Start and validate PITR result

```
kubectl -n "$NS" delete pod pgbackrest-recovery-toolbox --wait=true
kubectl -n "$NS" scale statefulset/"$REC_STS" --replicas=1
kubectl -n "$NS" rollout status statefulset/"$REC_STS" --timeout=10m
REC_POD=$(kubectl -n "$NS" get pod -l cluster-name=acid-pgbackrest-recovery -o jsonpath='{.items[0].metadata.name}')
kubectl -n "$NS" exec "$REC_POD" -- psql -U postgres -d banking -c "SELECT * FROM dr_lab_events ORDER BY id;"
kubectl -n "$NS" exec "$REC_POD" -- psql -U postgres -c "SELECT pg_is_in_recovery();"
kubectl -n "$NS" exec "$REC_POD" -- psql -U postgres -c "SELECT timeline_id FROM pg_control_checkpoint();"
```

Expected: rows committed before TARGET_TIME are present, the accidental DELETE is absent, and pg_is_in_recovery() is false after promotion.

13. Patroni Failover Considerations During Backup

Rediscover the current primary before each critical backup or archive action. A stale PRIMARY variable can point to a former primary after switchover/failover.

```
PRIMARY=$(for p in $(kubectl -n "$NS" get pod -l cluster-name="$CLUSTER" -o name | cut -d/ -f2); do
role=$(kubectl -n "$NS" exec "$p" -- curl -s http://127.0.0.1:8008/ | jq -r '.role'); if [ "$role"
= "primary" ] || [ "$role" = "master" ] || [ "$role" = "leader" ]; then echo "$p"; fi; done | head
-1)
echo "Current primary: $PRIMARY"
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "SELECT pg_switch_wal();"
kubectl -n "$NS" exec "$PRIMARY" -- sh -lc 'export PGBACKREST_REPO1_CIPHER_PASS="$(cat
/etc/pgbackrest-secret/repo-cipher-pass)"; pgbackrest --stanza=postgres-ha check; pgbackrest --
stanza=postgres-ha info'
```

14. Validation, Monitoring, Troubleshooting and Cleanup

14.1 Daily validation

```
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -x -c "SELECT * FROM pg_stat_archiver;"
kubectl -n "$NS" exec "$PRIMARY" -- sh -lc 'export PGBACKREST_REPO1_CIPHER_PASS="$(cat
/etc/pgbackrest-secret/repo-cipher-pass)"; pgbackrest --stanza=postgres-ha check; pgbackrest --
stanza=postgres-ha info'
kubectl -n "$NS" exec "$PRIMARY" -- patronictl list
```

14.2 Cleanup only recovery objects

```
kubectl -n "$NS" delete postgresql acid-pgbackrest-recovery --ignore-not-found
kubectl -n "$NS" delete pod pgbackrest-recovery-toolbox --ignore-not-found
kubectl -n "$NS" get pvc | grep acid-pgbackrest-recovery || true
```

Do not delete acid-postgres-ha PVCs or the pgBackRest repository as part of generic cleanup.

15. RPO / RTO and Suggested Backup Schedule

- WAL archive: continuous; determines how close to the incident you can recover.
- Full: weekly baseline/anchor, adjusted to database size and RTO.
- Differential: daily if you want fewer restore dependencies than a long incremental chain.
- Incremental: every 4-6 hours or as workload/RPO requires.
- Logical `pg_dump`: daily/weekly for critical databases when object-level recovery is useful.
- Restore drill: at least monthly and after major PostgreSQL/Spilo/pgBackRest/storage/IAM/operator changes.

15.1 Backup commands

```
pgbackrest --stanza=postgres-ha --type=full backup
pgbackrest --stanza=postgres-ha --type=diff backup
pgbackrest --stanza=postgres-ha --type=incr backup
```

16. Final Coverage Checklist

Topic	Status
Logical backup	Covered
Physical backup with <code>pg_basebackup</code>	Covered - corrected host/user/HBA/secret flow
WAL archiving	Covered
PITR	Covered
<code>pgBackRest</code> configuration	Covered
Repository	Covered
Compression	Covered
Encryption	Covered
Incremental backup	Covered
Differential backup	Covered
Retention policies	Covered
Full backup	Covered
Restore	Covered on isolated recovery storage
Patroni failover handling	Covered
Copy/paste-friendly commands	Every command is a separate paragraph

Most important corrections in this edition: `pg_basebackup` now uses `acid-postgres-ha` plus the `replication/standby` Secret instead of `127.0.0.1 + postgres`; the latest `pgBackRest` restore omits the invalid `--set=latest` pattern; recovery toolbox inherits the database pod service account for object-storage access; and every shell command is isolated for reliable copy/paste.